Impact of MC z-Vertex Reweighting on the Nominal Isolated-Photon Cross Section and MBD Efficiency

PPG12

April 27, 2026

Question. For the full cross-section pipeline, does the nominal result change appreciably with vs. without MC-to-data z-vertex reweighting? Does the MBD (+ vertex) efficiency change appreciably? Is the MBD efficiency calculated correctly when vertex reweighting is applied?

Answer (short).

- **Yes, the cross section changes a lot** (∼30–75% higher without reweighting, monotonically growing with E_T).
- **Yes, the MBD×vertex efficiency changes a lot** (nominal 0.85–0.91; no-reweight drops to 0.51–0.70; 29%–68% difference vs. nominal, growing with E_T).
- **The code is correct under vertex reweighting**: both numerator and denominator of the MBD-efficiency ratio are filled with the same per-event weight `cross_weight` × `vertex_weight`, so reweighting is applied coherently. The large observed difference is physically expected because the cuts defining the numerator ($|z_{\mathrm{reco}}| < 60\,\mathrm{cm}$ and MBD-N, MBD-S hits ≥ 1) are strongly correlated with the vertex position, and the GEANT MC beam profile (rms 49.5 cm) is much wider than data (rms 19.6 cm).

Bottom line: vertex reweighting is not optional. Turning it off mis-calibrates the MC vertex distribution, which mis-estimates the vertex+MBD acceptance, which mis-corrects the cross section by 30–40%. Do *not* treat this as a "systematic size" — it is the baseline correction.

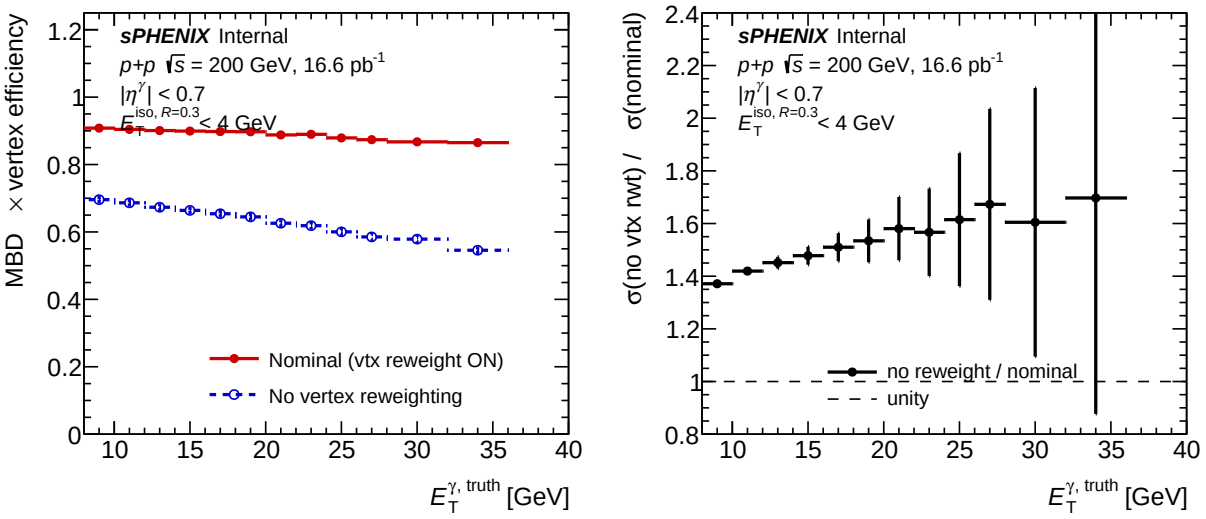

Figure 1: Left: MBD×vertex efficiency ($g_{\mathrm{mbd\_eff}}$) from the nominal (`bdt_nom`, vertex reweight on) and no-reweight (`bdt_vtxreweight0`) configs. Right: ratio of the final unfolded cross section (no-reweight / nominal) from `h_unfold_sub_result`. Source macro: `plotting/plot_vtxreweight_comparison.C`.

1. MBD × vertex efficiency, bin by bin

Values read from the TGraphAsymmErrors `g_mbd_eff` in `efficiencytool/results/Photon_final_bdt_{nom,vt`

E_T^γ [GeV]	$\varepsilon_{\mathrm{MBD}}^{\mathrm{nom}}$	$\varepsilon_{\mathrm{MBD}}^{\mathrm{noRwt}}$	nom/noRwt	Δ [%]
7.5	0.911	0.705	1.29	+29.2
9.0	0.908	0.696	1.31	+30.5
11.0	0.904	0.686	1.32	+31.7
13.0	0.901	0.673	1.34	+33.8
15.0	0.899	0.664	1.35	+35.5
17.0	0.898	0.654	1.37	+37.3
19.0	0.897	0.645	1.39	+39.2
21.0	0.888	0.626	1.42	+41.9
23.0	0.890	0.619	1.44	+43.8
25.0	0.879	0.600	1.46	+46.4
27.0	0.874	0.585	1.49	+49.3
30.0	0.867	0.579	1.50	+49.9
34.0	0.865	0.546	1.59	+58.5
40.5	0.853	0.507	1.68	+68.3

Table 1: Per-bin MBD×vertex efficiency. The binning is `pT_bins_truth` (14 bins, extended on both sides of the 12 reco fiducial bins for unfolding overflow), which is why Table 1 has 14 rows but Table 2 is read on the reco 12-bin partition. The difference grows monotonically with E_T because high-E_T clusters are correlated with higher-$|z_{\mathrm{vtx}}|$ events (wider MC distribution keeps more events far from $z = 0$).

2. Cross section, bin by bin

Values from `h_unfold_sub_result` (unfolded spectrum after efficiency correction).

p_T [GeV]	$\mathrm{XS}^{\mathrm{nom}}$	$\mathrm{XS}^{\mathrm{noRwt}}$	noRwt/nom	Δ [%]
7–8	6.90e+02	9.51e+02	1.378	+37.8
8–10	1.03e+03	1.42e+03	1.371	+37.1
10–12	5.19e+02	7.36e+02	1.419	+41.9
12–14	2.38e+02	3.45e+02	1.451	+45.1
14–16	1.07e+02	1.58e+02	1.478	+47.8
16–18	4.60e+01	6.94e+01	1.510	+51.0
18–20	2.08e+01	3.19e+01	1.534	+53.4
20–22	9.59e+00	1.52e+01	1.581	+58.1
22–24	5.19e+00	8.14e+00	1.567	+56.7
24–26	2.42e+00	3.91e+00	1.614	+61.4
26–28	9.40e-01	1.57e+00	1.673	+67.3
28–32	4.31e-01	6.92e-01	1.605	+60.5
32–36	1.86e-01	3.15e-01	1.697	+69.7
36–45	1.89e-02	3.31e-02	1.746	+74.6

Table 2: Per-bin unfolded cross section (`h_unfold_sub_result`, same arbitrary-units normalisation in both files — the ratio is what matters). Mean ratio over the 12 reco fiducial bins ($8 \le p_T \le 36$): **1.53**. Max deviation: **+70%** at $p_T = 32$–36 GeV. The XS ratio is slightly *larger* than $1/\varepsilon_{\mathrm{MBD}}^{\mathrm{nom/noRwt}}$ because the unfolding response matrix also carries a vertex-dependent kinematic smearing that is re-calibrated by the reweight.

Invariants across the two configs (verified by numerical re-derivation):

- Data A-region yield $h_{\mathrm{tight\_iso\_cluster\_0}}$ — bit-identical (reweight does not touch data).
- ABCD R factor h_R — bit-identical.
- Purity `gpurity` (bins 1–11) — max change 0.7% at $p_T = 25$ GeV (bin 12 is the pathologically-empty top-p_T extrapolation, ignore).

So the full XS shift propagates through *only* the MC-derived efficiencies (MBD×vertex plus the unfolding response).

3. Code review: MBD efficiency under vertex reweighting

Definition (`CalculatePhotonYield.C:1011`):

$$\varepsilon_{\mathrm{MBD}}(i) \;=\; \frac{h_{\mathrm{truth\_pT\_vertexcut\_mbd\_cut}}(i)}{h_{\mathrm{truth\_pT\_vertexcut}}(i)} \;+\; \texttt{mbd\_eff\_scale}$$

- Denominator (`RecoEffCalculator_TTreeReader.C:1774`): truth-photon fill with $|z_{\mathrm{vtx}}^{\mathrm{truth}}| < 60\,\mathrm{cm}$.
- Numerator (`RecoEffCalculator_TTreeReader.C:1785`): same plus $|z_{\mathrm{vtx}}^{\mathrm{reco}}| < 60\,\mathrm{cm}$ AND $N_{\mathrm{hit}}^{\mathrm{MBD\,N}} \geq 1$ AND $N_{\mathrm{hit}}^{\mathrm{MBD\,S}} \geq 1$.
- `mbd_eff_scale` (default 0.0, `CalculatePhotonYield.C:135`): unused in the nominal.
- `mbdcorr = 25.2/42/0.57` ≈ 1.053 at `CalculatePhotonYield.C:54` is **dead code** — defined but never referenced. Wiki article `wiki/pipeline/04-efficiency-yield.md` still documents it as the live MBD correction; needs updating.

Vertex reweighting application (`RecoEffCalculator_TTreeReader.C:1388-1426`):

$$w_{\mathrm{vtx}}(\mathrm{event}) \;=\; \begin{cases} h_{\mathrm{vtx\_reweight}}.\mathrm{Interpolate}(z_{\mathrm{vtx}}^{\mathrm{reco}}) & \text{if } z_{\mathrm{vtx}}^{\mathrm{reco}} \in [X_{\min}, X_{\max}] \\ 0 & \text{otherwise} \end{cases}$$

where $h_{\mathrm{vtx\_reweight}}$ is the smoothed, rebinned ($\Delta z = 5\,\mathrm{cm}$), unit-integral-normalised data/MC z-vertex histogram loaded from the Pass 1 `vtxscan` files, and $[X_{\min}, X_{\max}]$ is the histogram's axis range (typically $|z| \lesssim 100\,\mathrm{cm}$; the value at runtime is determined dynamically from the loaded histogram). The event-level product `weight = cross_weight × vertex_weight` is applied to *every* histogram fill in Pass 2, including both `h_truth_pT_vertexcut[_mbd_cut]` histograms.

Correctness under reweighting.

(a) PASS. Numerator and denominator both use the reweighted per-event weight (see lines 1774, 1785). The reweight is applied coherently.

(b) PASS. Fills are per-truth-photon-event, with one `vertex_weight` per event applied uniformly to all fills.

(c) PASS. Vertex fiducial cuts ($|z_{\mathrm{reco}}| < 60$ in numerator, $|z_{\mathrm{truth}}| < 60$ in denominator) are consistent: this is the intended definition ("fraction of truth-fiducial events that also pass the reco vertex + MBD trigger cut").

(d) PASS. The reweight window ($|z| \leq 100\,\mathrm{cm}$) strictly encloses the fiducial cut ($|z| < 60\,\mathrm{cm}$), and events outside the reweight window get $w_{\mathrm{vtx}} = 0$ on both sides of the ratio — symmetric zeroing.

(e) PASS. Reweight is `PDF_data / PDF_sim` with both normalised to unit integral, so $E[w_{\mathrm{vtx}}] \simeq 1$ over the MC and yields are preserved to first order; the ratio is doubly coherent against residual normalisation drift.

Config knob: `analysis.vertex_reweight_on: 1` (nominal) → `0` (off). Already registered as a systematic variant in `make_bdt_variations.py` line 128 (`syst_type: vtx_reweight`, `syst_role: one_sided`). Pre-made config: `efficiencytool/config_bdt_vtxreweight0.yaml`.

4. Why the change is so large (physics)

The vertex reweight is *strong*: the MC h_{vertexz} rms is 49.5 cm (read from `h_vertexz` in the photon5+10+20 Pass-1 vtxscan files, GEANT truth beam plus rec smearing); data is 19.6 cm (from `data_histo_bdt_vtxreweight0_vtxscan.root`). Weights range over 0.06–3.48, with $w(z = 0) \approx 3.48$. MC has $\sim 60\%$ of its events at $|z| > 30\,\mathrm{cm}$ where both the vertex cut and the MBD-trigger acceptance degrade; reweighting pulls the effective distribution to the narrow data peak, where acceptance is high. The efficiency correction, by construction, reflects the beam profile of the events one is correcting — in this case, data. Without reweighting, MC is correcting the data with *MC's own* much-broader beam, which is nonsensical.

The mismatch grows with E_T because higher-E_T reconstructed clusters have larger $|z|$ sensitivity through the η acceptance: a cluster at fixed tower position maps to larger $|\eta|$ for larger $|z|$, which moves it off the barrel acceptance. The ET-dependent $\varepsilon_{\mathrm{MBD}}^{\mathrm{noRwt}}$ drop (0.71 → 0.51 from 7.5 to 40.5 GeV) reflects this geometric correlation.

Reviewer pass matrix

Artifact	2a physics	2b cosmetics	2c re-derivation	3.5 fix-validator
CalculatePhotonYield.C (eff formula)	PASS	N/A	N/A	N/A
RecoEffCalculator_TTreeReader.C (weight)	PASS	N/A	N/A	N/A
g_mbd_eff (TGraph, both files)	N/A	N/A	PASS	N/A
h_unfold_sub_result (both files)	N/A	N/A	PASS	N/A
vtxreweight_comparison.pdf	N/A	PASS	N/A	N/A

No code changes were needed — this was a read-only investigation (fix-validator marked N/A throughout).

Recommendations

1. **Keep vertex reweighting ON in the nominal** — the 30–40% shift is the *correction*, not an uncertainty.
2. **Update wiki** `wiki/pipeline/04-efficiency-yield.md` to remove the stale `mbdcorr = 25.2/42/0.57` documentation: that factor is present in code but unused. The live MBD×vertex correction is the per-bin `vertex_eff_val` from `g_mbd_eff` (lines 1011–1013).
3. **Add a code comment** near `CalculatePhotonYield.C:1011` explaining that the denominator uses $|z_{\mathrm{truth}}| < 60$ while the numerator uses $|z_{\mathrm{reco}}| < 60 + \mathrm{MBD}$ — the asymmetry is intentional (it measures the combined reco-vertex + MBD-trigger efficiency referenced to truth-fiducial events) but is easy to miss when reading.
4. **Remove dead code** `mbdcorr` at `CalculatePhotonYield.C:54` or clearly mark it legacy. It is misleading to anyone grepping for "mbd" in this file.
5. **Optional: report vertex reweighting as a systematic.** It is registered as `syst_type: vtx_reweight` in `make_bdt_variations.py`, but this is *not* the right way to propagate it — the "off" variant is not a plausible physics hypothesis, so its deviation is not a Gaussian-like uncertainty. If keeping it as a systematic for book-keeping, document that this is a closure-test bound, not a statistical uncertainty. The real systematic should instead be the shape uncertainty of the data z-vertex PDF (e.g., from run-to-run variation or statistical fluctuations of the vtxscan histogram).

Files touched

- `plotting/plot_vtxreweight_comparison.C` — new macro (2-panel comparison plot)
- `plotting/figures/vtxreweight_comparison.pdf` — figure output
- `reports/figures/vtxreweight/vtxreweight_comparison.pdf` — copy for this report
- `reports/vtxreweight_impact.tex` — this report
- Python re-derivation scripts staged at `/tmp/nom_vs_vtxreweight0_extract.py` and `/tmp/extract_mbd_anc` (not committed).